esprit
Se former autrement

Séquence de démarrage Linux

Année Universitaire
2022-2023

Objectifs

- Maitriser la séquence de démarrage des systèmes d'exploitation Linux.
- Configurer le gestionnaire de démarrage.
- Comprendre le processus d’initialisation.
- Personnaliser le processus d’initialisation.

Plan

- Partie 1: Séquence de démarrage
- Partie 2: Processus d’initialisation

Introduction

- Suite à la mise sous tension de l'ordinateur, la séquence de démarrage est déclenchée, qui comporte principalement les étapes suivantes:
  - Localiser et lancer le chargeur de démarrage ,
  - Exécuter le noyau du système d'exploitation,
  - Lancer le premier processus

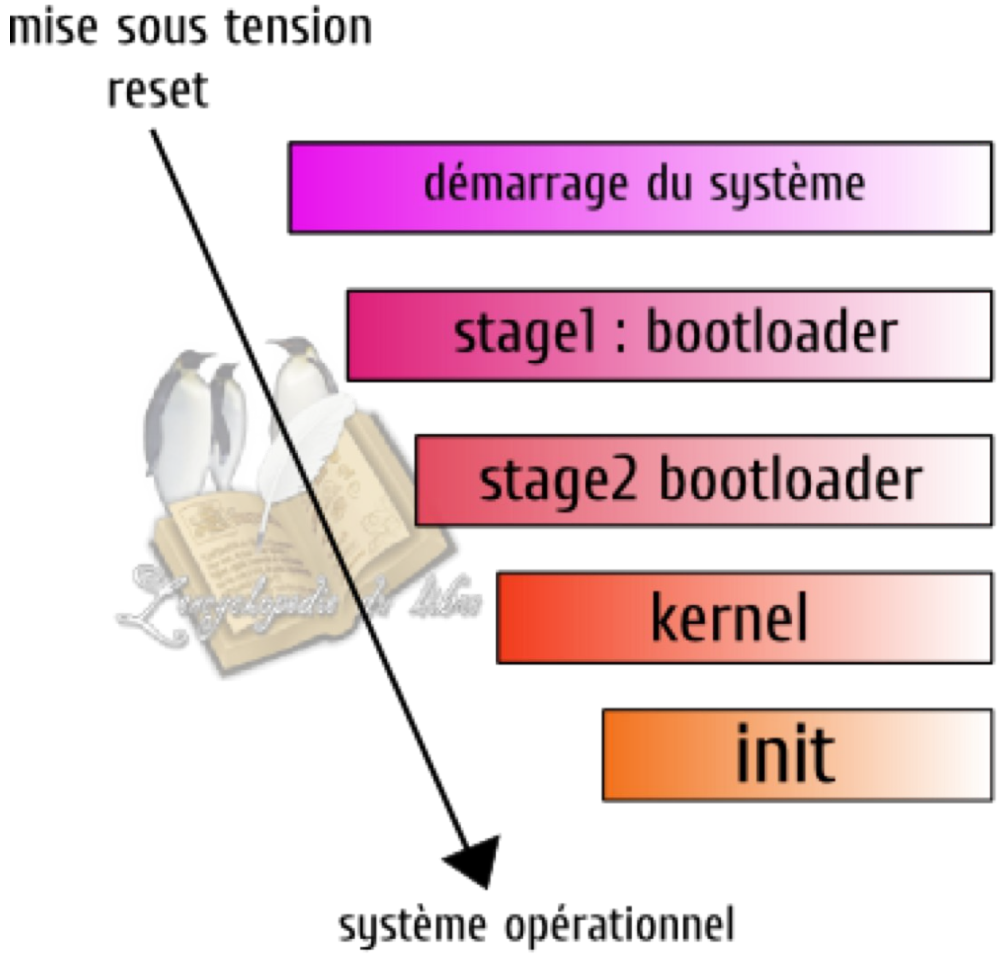
Séquence de démarrage

Gestionnaire de démarrage

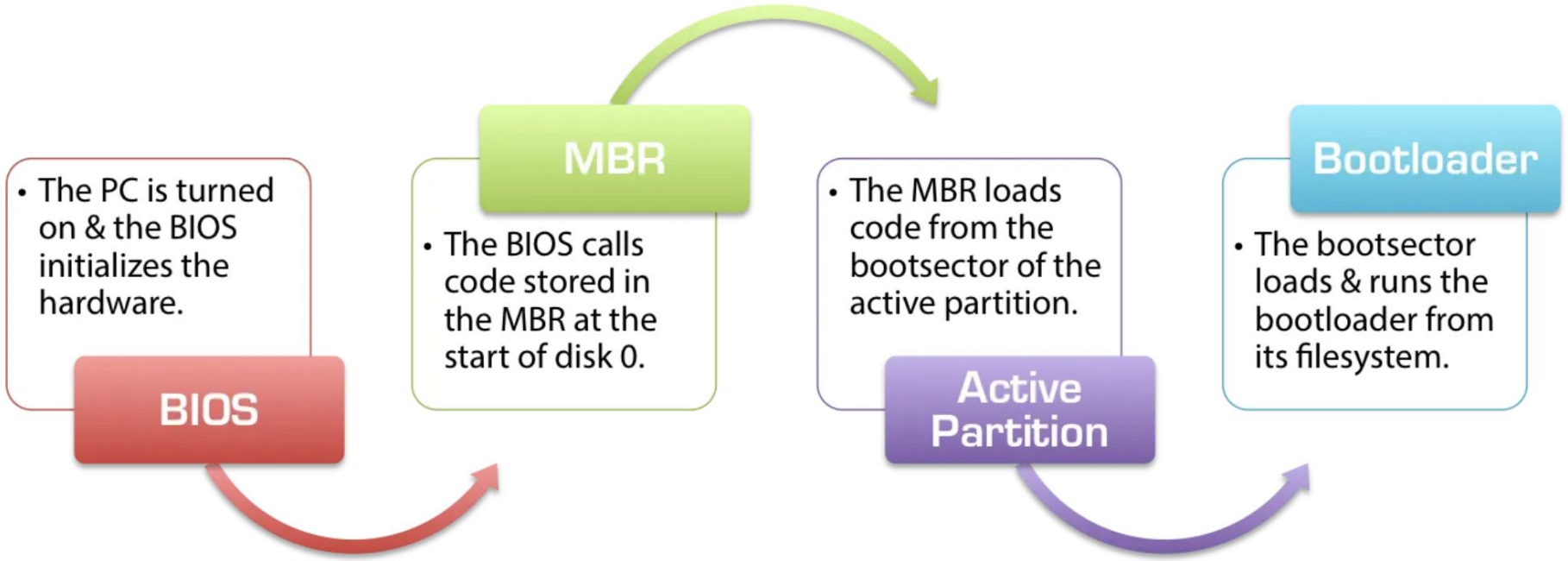
Les gestionnaires de démarrage, responsables du lancement du noyau, ont bien évolué. Cette évolution permet de combler les limites des anciennes versions et de proposer des nouvelles fonctionnalités.

Séquence de démarrage

Comment c'était?



Séquence de démarrage

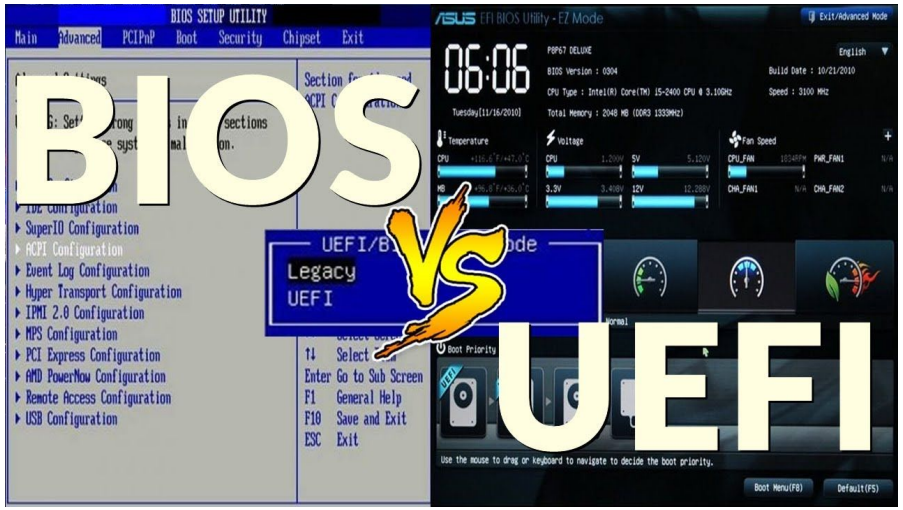


Limites MBR & BIOS

- La table de partitionnement du **MBR** possède des limitations:
  - 4 partitions primaires maximum.
  - Taille d'une partition limitée à 2.2 To.
- Le **BIOS** ne dispose que d'1 Mo d'espace mémoire pour s'exécuter. Il a ainsi du mal à initialiser plusieurs périphériques en même temps, ce qui ralentit le démarrage du PC

Naissance de l'UEFI

- UEFI « Unified Extensible Firmware Interface » remplace le BIOS depuis 2006. Il existe de nombreux avantages comparés à son ancêtre, le BIOS. L'UEFI permet :
  - Prise en charge des disques supérieurs à 2,2To (jusqu'à 9,4 Zo)
  - Prise en charge du Secure boot
  - Démarrage plus rapide
  - Mécanisme de rétro-compatibilité
  - Prise en charge des fonctionnalités réseaux



UEFI:Définition

- l'UEFI est un firmware stocké dans **une mémoire flash** située sur la carte mère de l'ordinateur.
- UEFI est installé au moment de la fabrication et constitue le premier programme qui s'exécute lors du démarrage d'un ordinateur.
- UEFI détecte les périphériques candidats à démarrer
- Le système de fichiers pris en charge par UEFI est basé sur le système de fichiers FAT (FAT12, FAT16 et FAT32).
- L'une des fonctionnalités du micrologiciel UEFI est de pouvoir stocker les entrées de démarrage dans la mémoire NVRAM,
- La NVRAM , Non Volatile RAM, équivalent du CMOS des anciens BIOS.

UEFI: NVRAM

La NVRAM est une mémoire vive RAM non volatile utilisée par UEFI pour stocker les variables qui doivent persister entre les démarrages.

La NVRAM contient une liste de variables y compris les options de démarrage suivantes:

- ✔ Quel système est actuellement démarré ?
- ✔ L'ordre des entrées de démarrage
- ✔ Le timeout avant de démarrer sur l'entrée par défaut (suivant l'ordre précédent)
- ✔ La liste des entrées de démarrage

UEFI: NVRAM

Les variables NVRAM de type **EFI_LOAD_OPTION** sont une combinaison d'un GUID (Globally Unique Identifier) du propriétaire de la variable, d'un nom et d'une valeur.

GUID	Nom de la variable	Description générale
8BE4DF61-93CA-11D2-AA0D-00E098032B8C (Efi)	BootOrder	Le système essaie de démarrer à partir de chacune des entrées Boot#### dans l'ordre dans lequel ils sont répertoriés dans cette variable.
8BE4DF61-93CA-11D2-AA0D-00E098032B8C (Efi)	Boot####	Une entrée de démarrage particulière qui peut être amorcée. #### est un nombre hexadécimal à quatre chiffres.

Efibootmgr

EFIBootmgr est un utilitaire en ligne de commande permettant de gérer le chargeur de démarrage EFI, il permet de:

- Répertorier les entrées du gestionnaire de démarrage EFI
- Changer l'ordre des entrées
- Supprimer une entrée de démarrage
- Créer une entrée de démarrage
- Définir l'intervalle de temporisation du gestionnaire de démarrage EFI

Efibootmgr

La commande **efibootmgr -o** permet de changer l'ordre de démarrage

```
root@souha-virtual-machine:~# efibootmgr
BootCurrent: 0004
Timeout: 2 seconds
BootOrder: 0004,0000,0001,0002,0003
Boot0000* EFI VMware Virtual SCSI Hard Drive (0.0)
Boot0001* EFI VMware Virtual SATA CDROM Drive (1.0)
Boot0002* EFI Network
Boot0003* EFI Internal Shell (Unsupported option)
Boot0004* ubuntu
```

BootCurrent: indique l'entrée de démarrage utilisée pour démarrer le système. Dans notre cas, 0004 (Ubuntu).

Timeout: indique le temps avant l'amorçage. 2 secondes dans notre cas.

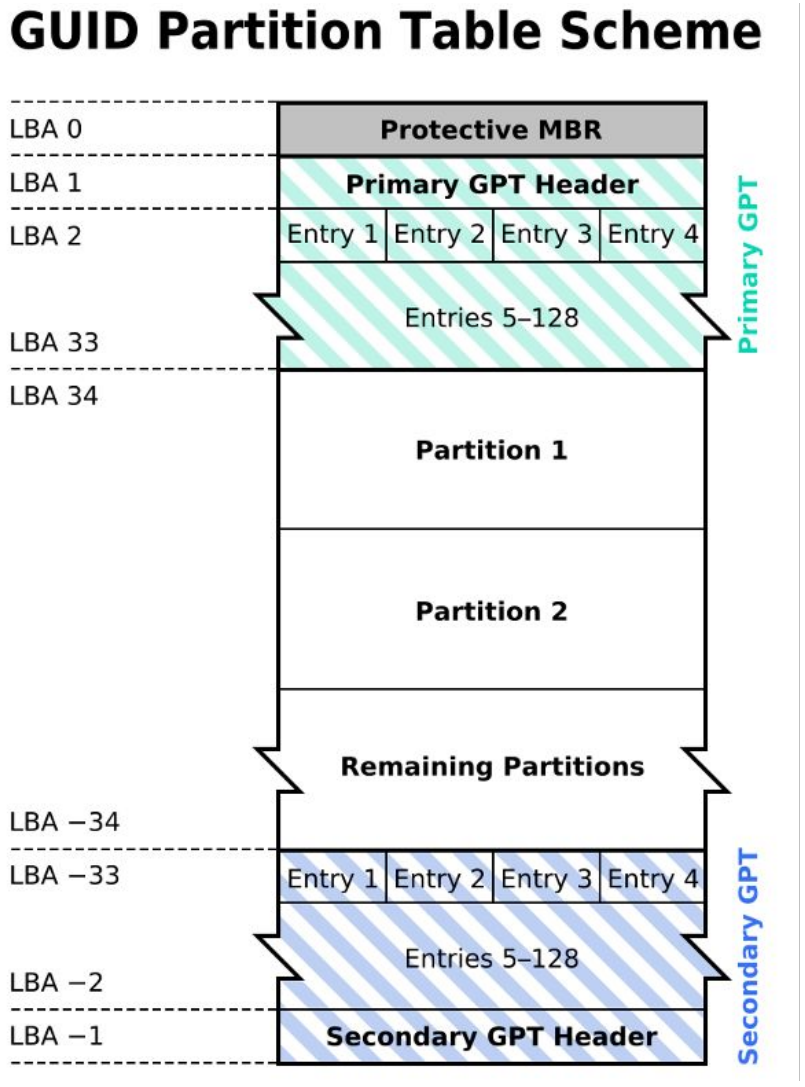
BootOrder: indique l'ordre dans lequel sont amorcés les systèmes UEFI.

Structure GPT

- UEFI utilise un schéma de partitionnement en **GPT « Globally Unique Identifier Partition Table »** au lieu du **MBR** utilisé par le BIOS. Ce qui permettra d'utiliser des disques système de plus de 9 Zettaoctets.
- **GPT** soit **G**lobally Unique Identifier **P**artition **T**able, est un standard pour décrire la table de partitionnement d'un disque dur.
- GPT utilise l'adressage logique des blocs (**LBA**) et non l'adressage historique **CHS** (cylinder-head-sector).

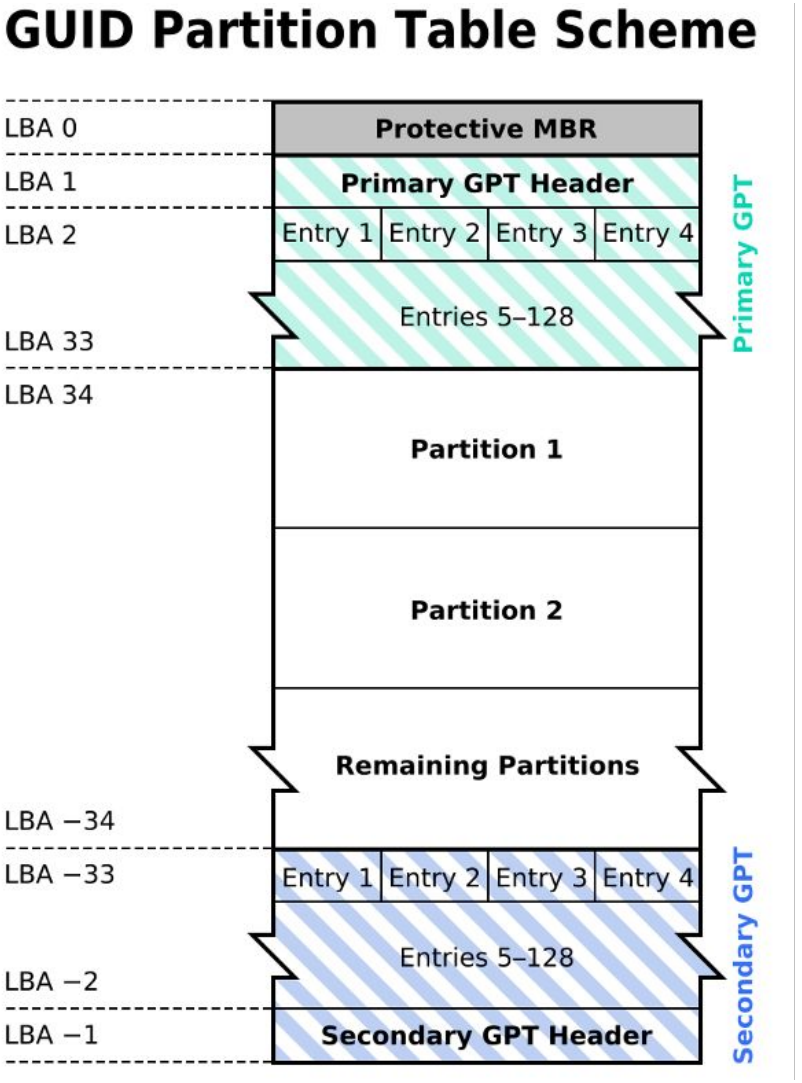
Structure GPT

- Sur un disque MBR, le MBR se trouve à l'adresse 0 (LBA). Dans les spécifications de GPT, à cette même adresse se trouve une structure qui **protège les disques GPT** des écritures provenant d'utilitaires disques qui ne (re)connaissent pas les informations de GPT appelée MBR protecteur (protective MBR).

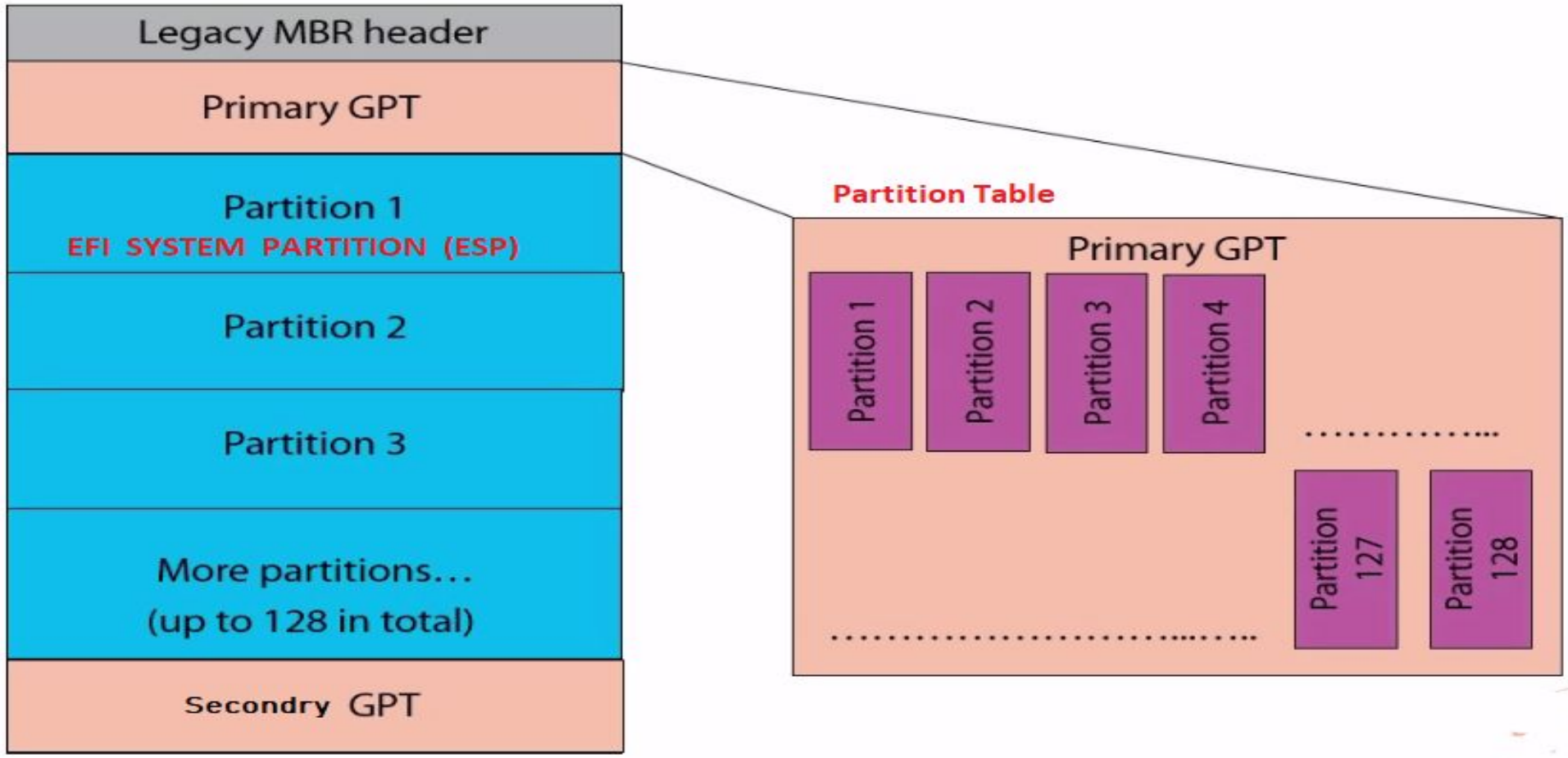


Structure GPT

- **L'entête de GPT** définit les blocs utilisables sur le disque ainsi que les **nombres** et la **taille** des descripteurs de la table de partitionnement.
- **Descripteurs de partitions (LBA 2 à 33 et -33 à -2)**: contiennent :
  - ✔ Le **type** de la partition,
  - ✔ **GUID: Globally Unique ID**
  - ✔ Les **adresses de début et de fin** de la partition.



Structure GPT

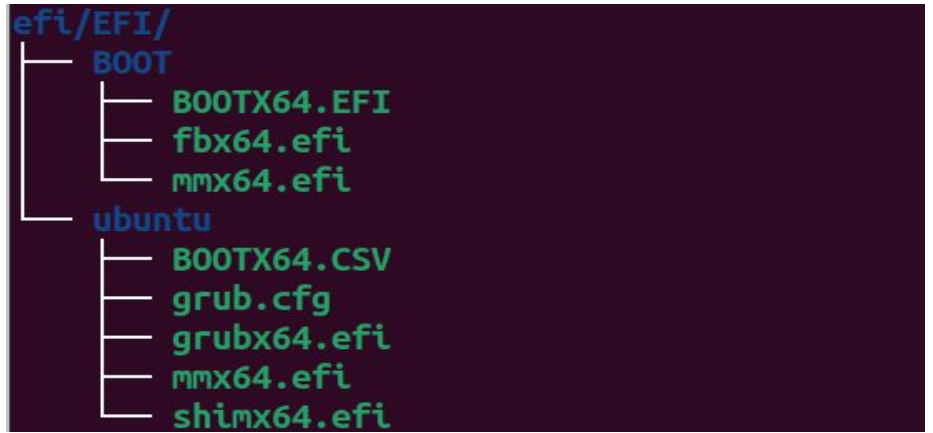


EFI System Partition (ESP)

La partition ESP contient tout le nécessaire pour le démarrage de système:

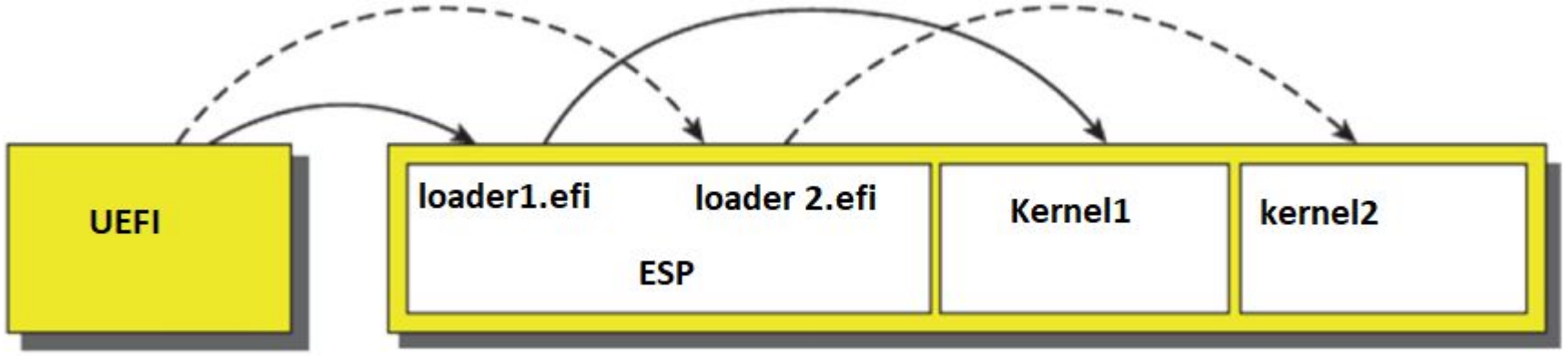
- **BOOTX64.EFI** : est le fichier du gestionnaire de démarrage (Boot Manager)
- **fbx64.efi :** est responsable de résoudre le problème des entrées NVRAM manquantes
- **shimx64.efi:** pour démarrer GRUB2 si le secure boot est active.
- **mmx64.efi:** est MOK manager (Machine Owner Key) qui sécurise le processus de démarrage.
- **grubx64.efi:** est le fichier du chargeur de démarrage (Boot Loader).
- **BOOTX64.CSV:** est le fichier lu lors de l'installation pour ajouter l'entrée de boot en NVRAM.

```
souha@souha-virtual-machine:/tmp$ sudo cat efi/EFI/ubuntu/grub.cfg
search.fs_uuid 21466f33-6a5d-44b6-b895-d8ee41f1dc9b root hd0,gpt3
set prefix=($root)'/boot/grub'
configfile $prefix/grub.cfg
```



EFI System Partition (ESP)

- L'UEFI localise le chargeur de démarrage GRUB2 qui est stocké dans la partition système ESP caractérisée par un GUID unique « *C12A7328-F81F-11D2-BA4B-00A0C93EC93B* »



Séquence de démarrage: Étapes

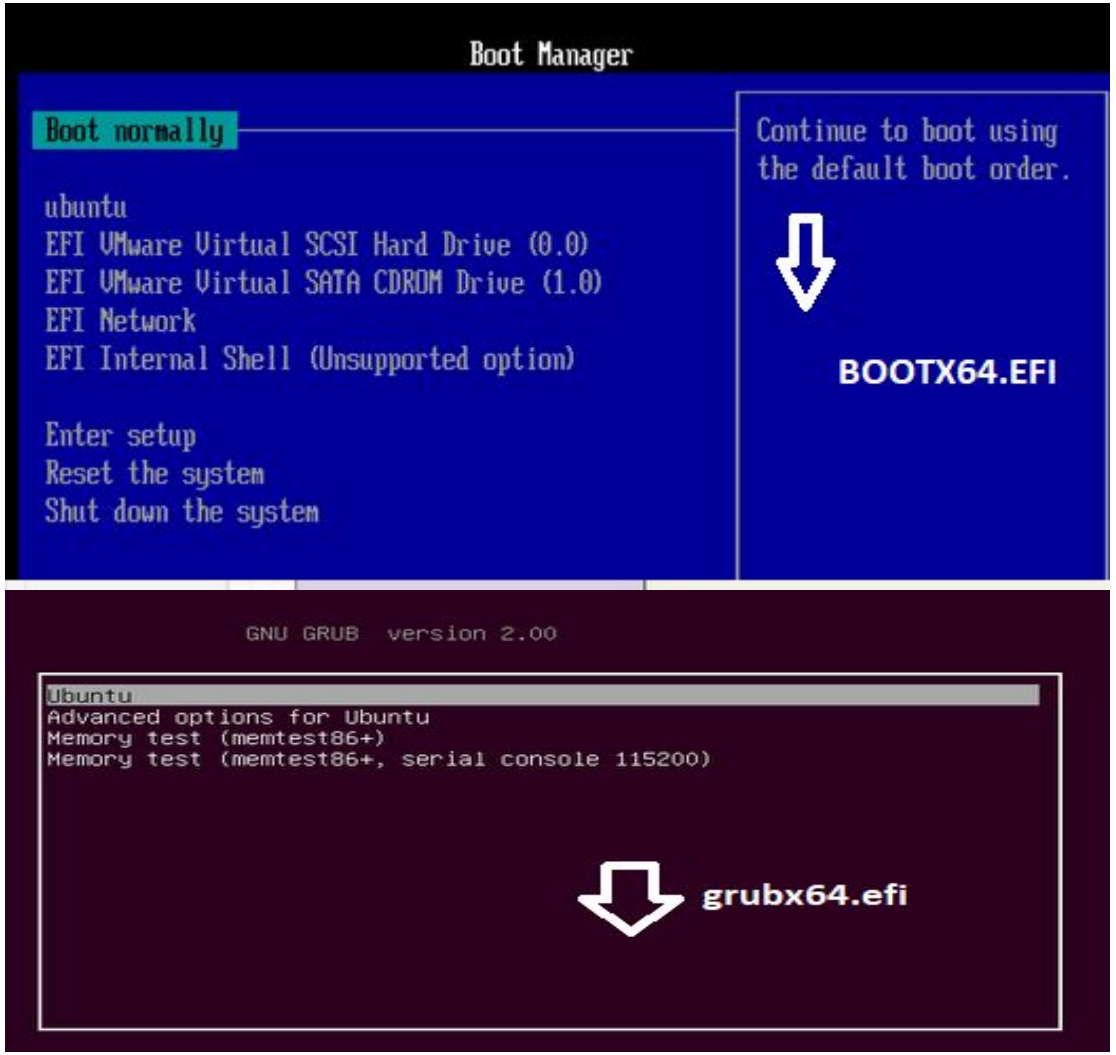
1. UEFI effectue un autotest du système (POST) pour identifier les défaillances matérielles élémentaires dès que la machine est mise sous tension.

2. UEFI active les composants basiques pour charger le système, comme la sortie vidéo, le clavier et les médias de stockage.

3. UEFI recherche une partition GPT avec un GUID spécifique qui l'identifie comme la partition système EFI (ESP). En cas de présence de plusieurs périphériques de démarrage, le gestionnaire de démarrage UEFI détermine l'ESP approprié à utiliser, en fonction de l'ordre défini dans le gestionnaire de démarrage (**boot manager**)

Séquence de démarrage: Étapes

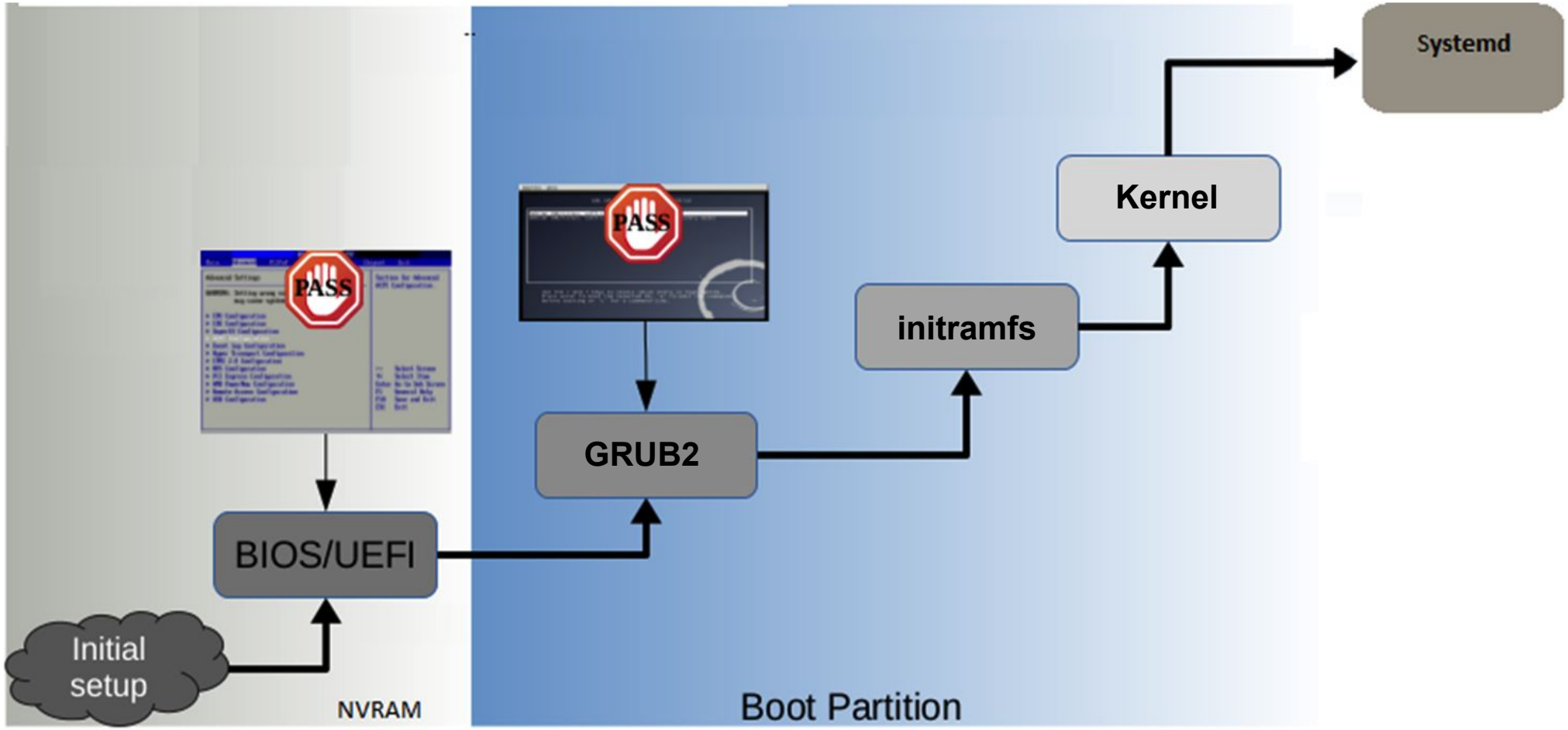
4. Si le démarrage sécurisé n'est pas activé, le gestionnaire de démarrage **BOOTX64.Efi** exécute le chargeur de démarrage **grubx64.efi.** Sinon l'application **shim.efi**, responsable de la certification, est chargée en premier avant le chargeur de démarrage GRUB. Si le certificat est valide, le chargeur de démarrage s'exécute et, à son tour, valide le noyau à charger.



Séquence de démarrage: Étapes

5. Le chargeur de démarrage charge le **vmlinuz**, fichier image du noyau, en mémoire et extrait le contenu du fichier image **initramfs**.
6. Le noyau charge les modules nécessaires à partir de l'**initramfs** pour accéder au système de fichiers racine.
7. Le noyau démarre le premier processus

Séquence de démarrage



Boot loader: GRUB 2

- GRUB dans sa version 2 est un chargeur de démarrage libre.
- GRUB2 est composé principalement de trois fichiers:
  1. ***/etc/default/grub*** un fichier contenant les paramètres du menu de GRUB 2,
  2. ***/etc/grub.d/*** un répertoire contenant les scripts de création du menu GRUB 2, permettant de personnaliser le menu de démarrage,
  3. ***/boot/grub/grub.cfg*** un fichier de configuration final de GRUB 2, non modifiable qui est généré automatiquement par le programme **grub-mkconfig** à partir des scripts **/etc/default/grub** et **/etc/grub.d/**

/etc/default/grub

- Le fichier **/etc/default/grub** contient les paramètres du menu GRUB qui sont lus et utilisés par les scripts sous ***/etc/grub.d/***, ce qui permet de générer le fichier ***grub.cfg***.

```
GRUB_DEFAULT=0
GRUB_TIMEOUT_STYLE=hidden
GRUB_TIMEOUT=0
GRUB_DISTRIBUTOR=`lsb_release -i -s 2> /dev/null || echo Debian`
GRUB_CMDLINE_LINUX_DEFAULT="quiet"
GRUB_CMDLINE_LINUX="find_preseed=/preseed.cfg auto noprompt priority=critical l
ocale=en_US"
```

/etc/default/grub

- ***GRUB_DEFAULT*** correspond à l'entré (OS) qui sera sélectionné par défaut. Une valeur de 0 démarre la première entrée de menu. Une valeur de 1 démarre la deuxième entrée de menu. Une valeur de « Saved » indique à GRUB 2 de charger le dernier système d'exploitation chargé avec succès. Pour le visualiser : *grub-editenv list*
- ***GRUB_TIMEOUT*** est la durée en secondes de l'affichage du menu avant de lancer le système sélectionné par défaut.
- ***GRUB_DISTRIBUTOR*** définit la syntaxe des titres du menu.

/etc/default/grub

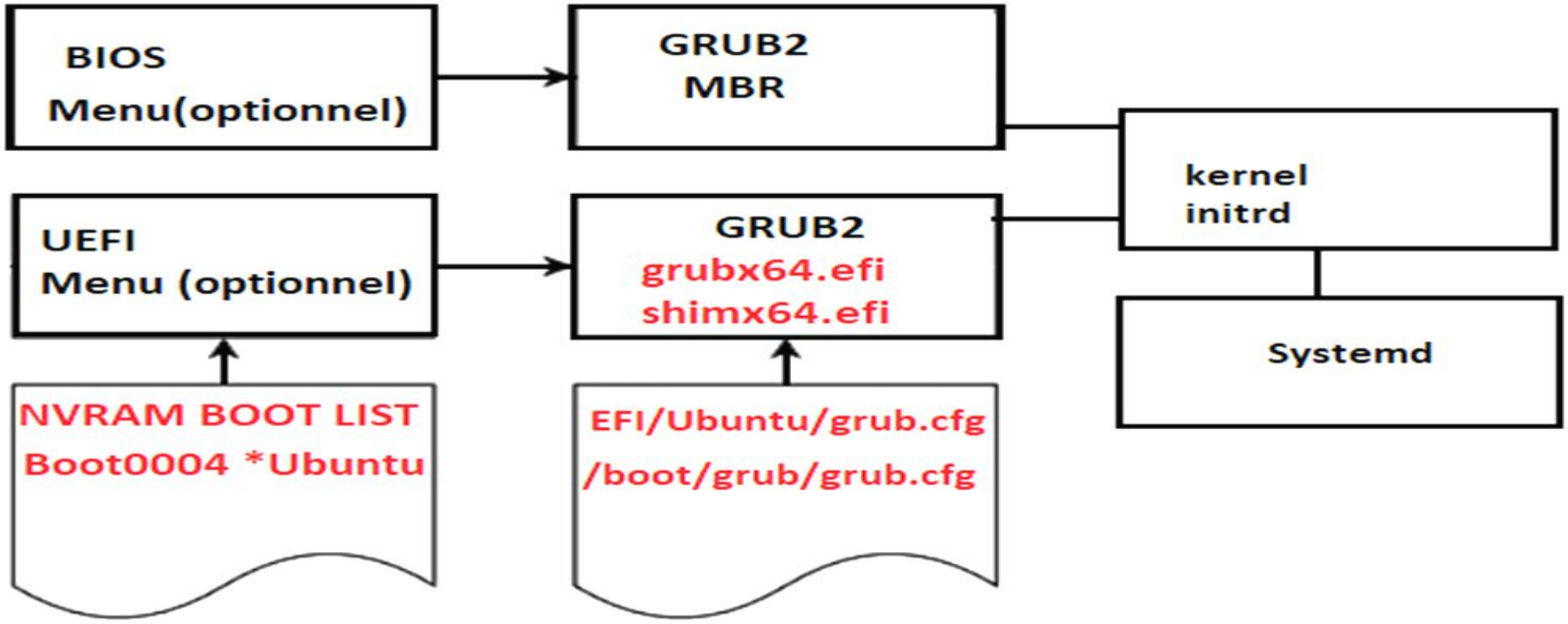
- ***GRUB_CMDLINE_LINUX_DEFAULT*** affichera simplement un écran vide avant d'afficher le menu GRUB.
- ***GRUB_CMDLINE_LINUX*** définie les options de noyau communes. Par exemple :
  - ✔ mem : Limite la quantité de la mémoire
  - ✔ quiet : Masque la plupart des messages de démarrage.
  - ✔ root: Définit la partition racine
  - ✔ Maxcpus: Limite le nombre de processeurs (ou cœurs de processeur)
  - ✔ …

/etc/grub.d/

- Le dossier /etc/grub.d contient tous les scripts qui seront utilisés pour créer le fichier grub.cfg.
  - **00_header** : script gérant les paramètres définis dans /etc/default/grub ;
  - **05_debian_theme** : script pour gérer le thème (fonds d'écran et couleurs) ;
  - **10_linux** : contient le script de lancement du système d'exploitation;
  - **20_memtest86+** : script permettant de générer les entrées memtest ; semble absent de certaines installations EFI;
  - **20_linux_xen** : script pour Xen Linux et Xen Hypervisor ;
  - **30_os-prober** : contient le script de recherche des autres systèmes installés ;
  - **30_uefi-firmware** : script pour localiser les paramètres uefi de l'ordinateur ;
  - **40_custom** : configuration personnelle (systèmes à lancer en configuration manuelle, paramètres qui n'existent pas pour /etc/default/grub,..);
  - **41_custom** : identique à 40_custom (si on le renomme par exemple 07_custom;

Séquence de démarrage: BIOS et UEFI

Le schéma suivant décrit la séquence de démarrage via BIOS et UEFI :



Processus d'initialisation

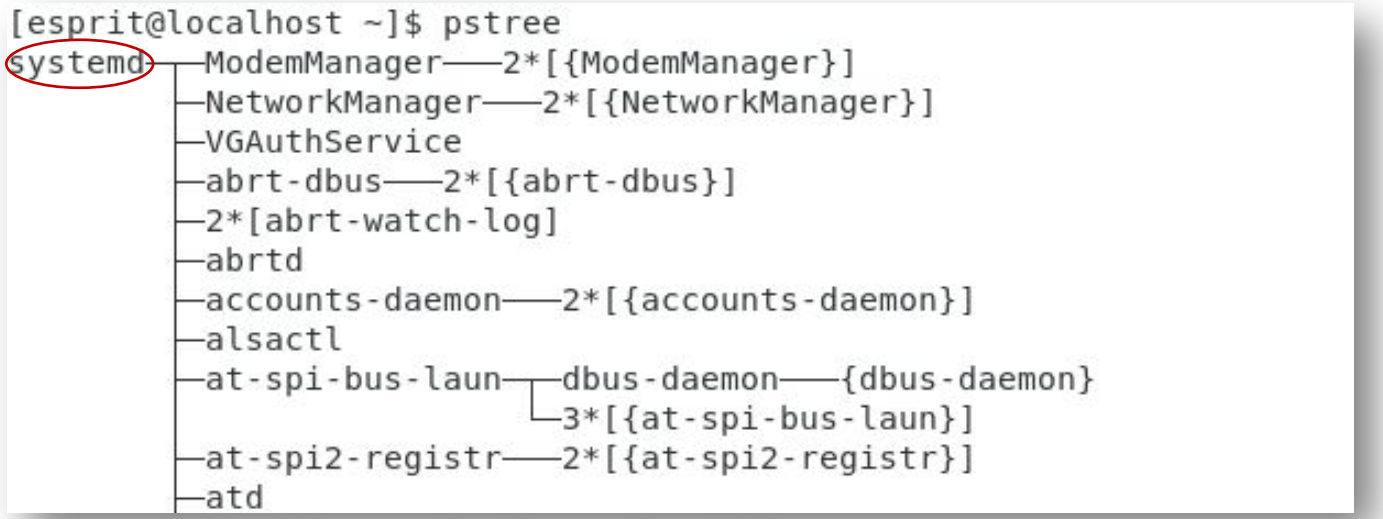
Processus d'initialisation

- Lorsque le chargeur de démarrage passe le contrôle au noyau du système, ce dernier se charge de:
  - Réserver la mémoire
  - Prendre en compte la zone d'échange (swap)
  - Détecter le matériel et charger les pilotes des périphériques
  - Monter le système de fichiers
  - **Lancer le 1er processus**

Processus d'initialisation: Systemd

Qu'est-ce que Systemd?

- **Systemd** (pour « **system daemon** » : le démon du système) est le système d'initialisation adopté par les dernières distributions Linux
- C'est un ensemble de programmes destines à la gestion système.



Processus d'initialisation: Systemd

- **Systemd** fournit de nombreuses fonctionnalités:
  - Le lancement en parallèle des services systèmes au démarrage du système,
  - L'activation à la demande de démons,
  - La gestion des dépendances entre les services,
  - …
- **Systemd** reste compatible avec les anciens scripts d'initialisation SysV,

Processus d'initialisation: Unités Systemd

- La plupart des actions de **systemd** est effectuée grâce à des unités (units) qui sont des ressources que **systemd** sait gérer. Les unités sont caractérisées par un nom et un type.

Type d'unité	Extension de fichier	Description
Unité du service	.service	Service système.
Unité cible	.target	Un groupe d'unités systemd.
Unité Automount	.automount	Un point Automount du système de fichiers.
Unité du périphérique	.device	Fichier du périphérique reconnu par le noyau.
Unité de montage	.mount	Point de montage du système de fichiers.
Unité de chemin	.path	Un fichier ou répertoire dans un système de fichiers.
Unité scope	.scope	Un processus créé de manière externe.
Unité de tranche	.slice	Un groupe d'unités organisées de manière hiérarchique qui gèrent des processus système.
Unité d'instantané	.snapshot	Un état enregistré du gestionnaire systemd.
Unité de socket	.socket	Un socket de communication inter-processus.
Unité swap	.swap	Un périphérique ou fichier swap.
Unité minuteur	.timer	Un minuteur systemd.

Processus d'initialisation: Structure de l'unité

[UNIT]

Description: – Cette déclaration fournit une brève description de l'unité.
Documentation: – Cette déclaration indique l'emplacement de la page de manuel (document d'aide) et la commande permettant d'accéder à cette page.
After: – Cette mention énumère les unités qui doivent être activées avant l'unite en question.
Before: – Cette déclaration énumère les unités qui doivent être activées après l'unite en question.
Wants: – Cette déclaration liste les unités/services qui doivent être démarrés avant l'unite en question.
Requires :- Cette déclaration énumère les unités/services qui doivent être obligatoirement démarrés avant l'unite en question.
Conflicts: – Cette déclaration énumère les unités/services qui doivent être arrêtés avant le démarrage de l'unite en question.

[TYPE]

Le titre de cette section (le mot clé entre les crochets) indique le type d'unité. Par exemple, si le type d'unité est le Service, cette section utilise le titre [Service] et si le type d'unité est le Socket, cette section utilise le titre [Socket]. Les déclarations de cette section contiennent les opérations de contrôle/commandes de l'unité.

[INSTALL]

Cette section contient des informations sur l'unité cible. L'option la plus courante que vous pouvez voir dans cette section est

WantedBy : - Nom de l'unité cible qui démarre automatiquement cette unité. Par exemple, si vous voyez multi-user.target dans cette déclaration, cela signifie que lorsque le système entre dans le niveau cible "multi-user.target", cette unité est démarrée automatiquement.

Processus d'initialisation: Structure de l'unité

/usr/lib/systemd/system/NetworkManager.service

```
[Unit]
Description=Network Manager
Documentation=man:NetworkManager(8)
Wants=network.target
After=network-pre.target dbus.service
Before=network.target

[Service]
Type=dbus
BusName=org.freedesktop.NetworkManager
ExecReload=/usr/bin/busctl call org.freedesktop.NetworkManager /org/freedesktop/NetworkManager org.freedesktop.NetworkManager Reload u 0
#ExecReload=/bin/kill -HUP $MAINPID
ExecStart=/usr/sbin/NetworkManager --no-daemon
Restart=on-failure
# NM doesn't want systemd to kill its children for it
KillMode=process

# CAP_DAC_OVERRIDE: required to open /run/openvswitch/db.sock socket.
CapabilityBoundingSet=CAP_NET_ADMIN CAP_DAC_OVERRIDE CAP_NET_RAW CAP_NET_BIND_SERVICE CAP_SETGID CAP_SETUID CAP_SYS_MODULE CAP_AUDIT_WRITE CAP_KILL CAP_SYS_CHROOT

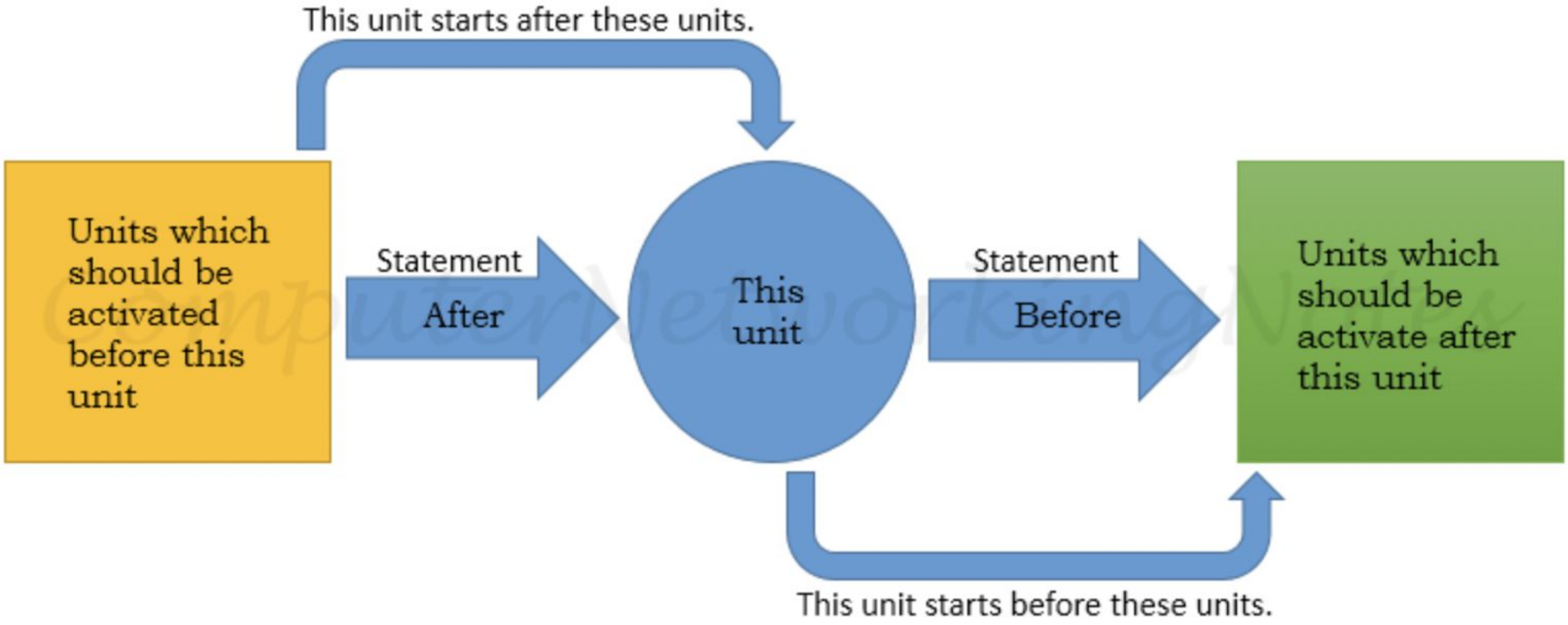
ProtectSystem=true
ProtectHome=read-only

# We require file descriptors for DHCP etc. When activating many interfaces,
# the default limit of 1024 is easily reached.
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
Also=NetworkManager-dispatcher.service

# We want to enable NetworkManager-wait-online.service whenever this service
# is enabled. NetworkManager-wait-online.service has
# WantedBy=network-online.target, so enabling it only has an effect if
# network-online.target itself is enabled or pulled in by some other unit.
Also=NetworkManager-wait-online.service
```

Processus d'initialisation: Structure de l'unité



Processus d'initialisation: Systemd

- Il identifie la cible par défaut du système à l'aide de **/usr/lib/systemd/system/default.target**
- Les fichiers de configuration de ces cibles sont sous **/usr/lib/systemd/system/**
- **Systemd** positionne ensuite le système dans l'état défini par la cible. Pour ce faire, il demarre, stoppe ou relance des unités de type service.

Processus d'initialisation: Systemd

/usr/lib/systemd/system/default.target

```
[Unit]
Description=Graphical Interface
Documentation=man:systemd.special(7)
Requires=multi-user.target
Wants=display-manager.service
Conflicts=rescue.service rescue.target
After=multi-user.target rescue.service rescue.target display-manager.service
AllowIsolate=yes
```

Processus d'initialisation: Targets Systemd

- Les unités de cible se terminent par l'extension **.target** et leur unique but consiste à regrouper d'autres unités **Systemd** dans une chaîne de dépendances,
- Par exemple, **l'unité graphical.target,** qui est utilisée pour lancer une session graphique, lance des services systèmes comme le gestionnaire d'affichage GNOME (**display-manager.service**) et active également l'unité **multi-user.target**.

Processus d'initialisation: Targets Systemd

/usr/lib/systemd/system/graphical.target

```
[Unit]
Description=Graphical Interface
Documentation=man:systemd.special(7)
Requires=multi-user.target
Wants=display-manager.service
Conflicts=rescue.service rescue.target
After=multi-user.target rescue.service rescue.target display-manager.service
AllowIsolate=yes
```

Processus d'initialisation: Targets Systemd

Tâches / Système	Systemd
Arrêt	poweroff.target
Mono utilisateur	rescue.target
Multi-utilisateur	multi-user.target
Multi-utilisateur avec réseau	multi-user.target
Personnalisable	runlevel4.target
Multi-utilisateur avec réseau et graphique	graphical.target
Reboot	reboot.target
Changer de runlevel (Ex 3)	systemctl isolate multi-user.target
Changer le runlevel par défaut (Ex 3)	systemctl set-default multi-user.target
Vérifier le runlevel actif	systemctl get-default

Systemd: Utilitaires de base

Tâches / Système	Systemd
Démarrer un service	systemctl start le service.service
Stopper un service	systemctl stop le service.service
Redémarrer un service	systemctl restart le service.service
Recharger un service	systemctl reload le service.service
Statut d'un service	systemctl status le service.service
Activer un service au démarrage	systemctl enable le service.service
Désactiver un service au démarrage	systemctl disable le service.service
vérifier si un service est activé au boot	systemctl is-enabled le service.service

Systemd: Utilitaires de base

Tâches / Système	Systemd
Arrêter le système	systemctl halt
Eteindre le système	systemctl poweroff
Redémarrer le système	systemctl reboot
Suspendre le système	systemctl suspend
Hiberner le système	systemctl hibernate
Afficher les logs systèmes	journalctl -f
Basculer temporairement de runlevel	systemctl isolate nom.target
Changer de runlevel par défaut	systemctl set-default nom.target

Résumé: Caractéristiques de Systemd

- Conception propre, moderne et efficace.
- Traitement simultané et parallèle au démarrage.
- Syntaxe d'unité simple.
- Empreintes mémoire réduites.
- Technique améliorée pour exprimer les dépendances.
- Instruction d'initialisation écrite dans le fichier de configuration et non dans le script shell.
- Enregistrement des événements avec **journald**. Choix de journalisation des événements du système avec systemd ainsi que syslog.
- L'état de systemd peut être préservé pour être appelé plus tard dans le futur.
- Suivi des processus en utilisant le **cgroup** du noyau et non le PID.